ECM2433 Mock Exam Paper 4

The C Family: C, C++, and Rust

Module: ECM2433

Time allowed: 2 Hours
Total marks: 100

Instructions

- Answer all questions.
- Marks for each part of a question are shown in brackets.
- Total marks for this paper: 100.

1 Question 1: Command-line Arguments, Function Pointers, and Build Tools (25 marks)

(a)

The `main` function in a C program can be written as `int main(int argc, char **argv)`.

(i) Explain the purpose of `argc` and `argv`. What is stored in `argv[0]`? [3 marks]

(ii) Write a short C code snippet to safely print the first command-line argument provided by the user (i.e., the first argument after the program name itself). If no arguments are provided, print an error message. [3 marks]

(b)

C supports function pointers, which are often used to pass functions as arguments to other functions (e.g., in `qsort` or custom sorting algorithms).

(i) Write the prototype for a function named `filter_array` that takes three arguments: an integer array, its length, and a function pointer. The function pointer should point to a function that takes an `int` and returns an `int`. [3 marks]

(ii) Explain one advantage of using function pointers to pass comparison logic rather than hardcoding the logic inside the sorting algorithm. [3 marks]

(c)

Dynamic memory allocation for a 2D array in C can be complex. Write a C function `int** allocate_2d(int rows, int cols)` that dynamically allocates a 2D array of integers using an array of pointers to rows. Ensure you handle the allocation correctly. [6 marks]

(d)

When compiling large C/C++ projects, developers often use `make` and a `Makefile`.

(i) What is the advantage of compiling source files into object files (`.o`) before linking them into an executable, rather than compiling all source files directly into an executable every time? [4 marks]

(ii) Briefly explain the role of `-Wall` and `-g` flags in the `gcc` compilation process. [3 marks]

2 Question 2: C++ Streams, Exceptions, and Linkage (25 marks)

(a)

You are writing a C++ program that needs to call a function `void calculate_stats(double* data, int n);` which was written and compiled in C.

(i) What keyword block must be used in your C++ code to allow it to link properly with the C object file? [2 marks]

(ii) Explain *why* this is necessary, referring to the concept of Name Mangling. [4 marks]

(b)

C++ streams (`std::cin`) manage their own error state.

(i) Suppose you use `std::cin >> my_integer;` but the user types `"Hello"`. What happens to the stream state, and what will happen to subsequent `std::cin` extraction attempts? [4 marks]

(ii) How does C++ stream formatting (e.g., using `std::hex` or `std::setprecision`) provide better type safety compared to C's `printf` format specifiers? [3 marks]

(c)

The C++ Standard Template Library (STL) provides powerful algorithms that can accept lambda expressions. Write a C++ code snippet that uses `std::count_if` and a lambda expression to count how many numbers in a `std::vector<int> numbers` are strictly greater than 50. [5 marks]

(d)

C++ introduces `try` and `catch` for exception handling.

(i) Write a simple C++ function `double divide(double a, double b)` that throws an exception of type `std::invalid_argument` if `b` is equal to 0.0. [4 marks]

(ii) Give one reason why throwing an exception might be preferable to returning a special error code (like `-1.0`) in this context. [3 marks]

3 Question 3: Rust Structs, Enums, and Shared Concurrency (25 marks)

(a)

Rust provides powerful pattern matching via the `match` statement and Enums.

```
enum Status {
    Pending,
    InTransit(String),
    Delivered,
}
```

Write a Rust function `fn print_status(s: &Status)` that uses `match` to print "Waiting" for `Pending`, "Arrived" for `Delivered`, and "Moving through [City]" for `InTransit`, extracting the inner string for the city. [5 marks]

(b)

In multi-threaded Rust programs, passing data between threads can be achieved using channels (`mpsc`).

(i) What does `mpsc` stand for? [2 marks]

(ii) If you need to send data from 3 different worker threads back to a single main thread, how do you handle the sender (`tx`) so that each thread has its own copy? [3 marks]

(c)

Smart pointers in Rust manage data differently depending on the use case.

(i) Explain the difference between `Box<T>` and `Rc<T>`. [4 marks]

(ii) Why will the Rust compiler prevent you from sending an `Rc<T>` across a thread boundary? [2 marks]

(d)

Rust iterators are highly expressive. Given a vector of strings:
`let words = vec![String::from("apple"), String::from("cat")];` Write a single chain of iterator methods (`iter()`, `map()`, `collect()`) that creates a new `Vec<usize>` containing the lengths of each string in `words`. [5 marks]

(e)

Rust supports "struct update syntax".

```
let p1 = Package { id: 1, dest: String::from("London"), weight: 2.0 };
let p2 = Package { id: 2, ..p1 };
```

After executing this code, a programmer tries to access `p1.dest` and receives a compiler error. Explain why this happens, referring to Rust's ownership rules and the `Copy` trait. [4 marks]

4 Question 4: Language Comparison: Polymorphism, Macros, and Thread Safety (25 marks)

(a) Polymorphism and Dynamic Dispatch

C, C++, and Rust all support ways to execute different code at runtime based on the type of data, but the mechanisms differ.

(i) In C++, how does the use of the `virtual` keyword enable dynamic polymorphism? Mention the role of the vtable. [4 marks]

(ii) In C, since there are no classes or inheritance, how can a programmer simulate dynamic dispatch (e.g., executing a different print function for different struct types)? [3 marks]

(b) Metaprogramming and Generics

(i) Explain the fundamental difference in how the compiler processes a C preprocessor `#define` macro versus a C++ `template`. [4 marks]

(ii) Give one reason why C++ templates or Rust generics are considered significantly safer to use than C macros when writing reusable code. [3 marks]

(c) Concurrency and Thread Safety

Writing multi-threaded programs in C/C++ is notoriously prone to "Data Races".

(i) What is a Data Race? [3 marks]

(ii) Explain how Rust's ownership model and strict mutability rules (e.g., only one mutable reference allowed at a time) effectively eliminate data races at compile time. [4 marks]

(d) Standard Library Philosophy

Briefly compare the philosophy of the standard library in C versus C++ (STL). Give one example of a high-level data structure provided out-of-the-box by C++ that requires manual implementation (or external libraries) in C. [4 marks]

End of Paper